

PASUL 1 DIN 5 / BACKEND 5A.1

Baza de date și înregistrarea utilizatorului

Tutorial pe codul real al proiectului Netex · Java 25 · Spring Boot · PostgreSQL

Ce vei putea explica la final

Cum ajunge un email și o parolă dintr-o cerere HTTP într-un rând din **users**, de ce parola devine hash, de unde vine ID-ul și de ce răspunsul API nu conține parola sau hashul.

Cum folosești ghidul fără să te copleșească

- Parcurge-l în **trei sesiuni de 25-30 de minute**: paginile despre SQL, apoi Java, apoi verificarea practică.
- Ține proiectul deschis în IntelliJ. La fiecare fragment din PDF, caută același loc în fișierul real.
- La finalul fiecărei sesiuni, închide PDF-ul și povestește traseul cererii în cuvintele tale. Nu încerca să memorezi toate adnotările.
- Semnul că ai înțeles: poți răspunde la întrebările de la final fără să citești explicația.

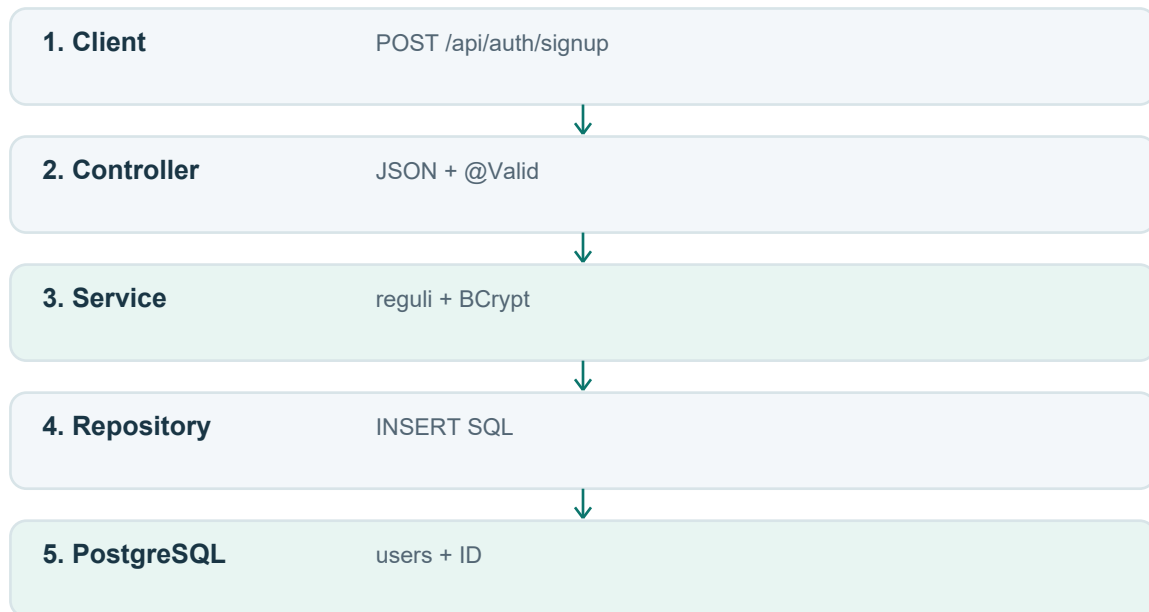
Cei cinci pași de învățare pe care îi păstrăm

Ordine	Ce studiem
1 · Acum	Baza de date și înregistrarea: HTTP → validare → hash → SQL → răspuns.
2	Loginul: ce citește codul nostru din SQL și ce verifică Spring Security.
3	Sesiunea, cookie-ul și tokenul CSRF.
4	Permisunile: public, utilizator logat și rol ADMIN.
5	Testele: urmărim o verificare completă și exersăm explicația de interviu.

În acest PDF studiem pasul 1. Loginul, sesiunea și CSRF apar numai cât este necesar ca să poți testa înregistrarea; le aprofundăm în pașii 2 și 3.

1. Mai întâi: harta întregii operații

Imaginează-ți cererea **POST /api/auth/signup** cu un corp JSON care are **email** și **password**. Serverul trebuie să creeze contul și să întoarcă un răspuns sigur.



Pe drumul înapoi, PostgreSQL dă ID-ul nou → Repository îl transmite către Service → Service creează un **AppUser** → Controller îl transformă în **UserResponse** → clientul primește HTTP 201 și JSON.

Cele 3 locuri pe care să le ții minte

Controller = intrare/ieșire HTTP. **Service** = regulile înregistrării. **Repository** = interacțiunea cu SQL. Baza de date verifică suplimentar regulile ei, inclusiv emailul unic.

Fișierele pe care le vom citi

Fișier	Responsabilitate
V1 și V2 (SQL)	Structura tabelului users și coloana role.
SignupRequest + EmailAddress	Datele primite și normalizarea emailului.
SignupController	Ruta HTTP și răspunsul 201.
SignupService	Validare suplimentară, hash, rol USER, eroarea 409.
UserRepository	INSERT parametrizat și ID-ul întors de SQL.
AppUser + UserResponse	Datele interne versus cele expuse clientului.

2. Ce există în PostgreSQL

users conține conturile care se pot autentifica. Nu este același lucru cu un contact din agendă. Un contact va avea, ulterior, un autor în **created_by_user_id**.

V1 a creat tabelul users

backend/src/main/resources/db/migration/V1__create_users_and_contacts.sql · liniile 1-11

```
1 CREATE TABLE users (  
2     id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
3     email VARCHAR(254) NOT NULL,  
4     password_hash VARCHAR(255) NOT NULL,  
5     created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
6     CONSTRAINT ck_users_email_trimmed CHECK (email = btrim(email) AND email <> ''),  
7     CONSTRAINT ck_users_password_hash_not_blank CHECK (btrim(password_hash) <> '')  
8 );  
9  
10 -- The database also prevents duplicate emails that differ only in letter case.  
11 CREATE UNIQUE INDEX uk_users_email ON users (lower(email));
```

- **id BIGINT ... IDENTITY PRIMARY KEY**: PostgreSQL generează un identificator unic. Java nu îl alege.
- **email VARCHAR(254) NOT NULL**: emailul este obligatoriu și are o limită de lungime.
- **password_hash**: coloana primește hashul BCrypt, niciodată parola introdusă.
- **created_at**: baza pune automat momentul creării dacă Java nu trimite o valoare.
- **uk_users_email** pe **lower(email)**: nu permite două conturi cu același email scris cu litere diferite.

Două tipuri diferite de reguli

Java validează pentru un mesaj util către utilizator. PostgreSQL protejează datele chiar dacă alt cod încearcă să insereze un rând greșit. Indexul unic este ultima apărare și la două înregistrări simultane.

V2 a adăugat rolul fără să schimbe V1

backend/src/main/resources/db/migration/V2__add_user_role.sql · liniile 1-3

```
1 ALTER TABLE users ADD COLUMN role VARCHAR(16) NOT NULL DEFAULT 'USER';  
2  
3 ALTER TABLE users ADD CONSTRAINT ck_users_role CHECK (role IN ('USER', 'ADMIN'));
```

DEFAULT 'USER' completează și rândurile create înainte de V2 și este valoarea implicită pentru un INSERT care omite rolul. **CHECK** permite doar USER sau ADMIN. Înregistrarea publică trimite explicit USER; o proprietate **role** din JSON nu este folosită.

3. Ce intră prin HTTP

Un exemplu de corp trimis de client este `{"email": "ANA@example.com", "password": "ParolaDeTest123!"}`. Spring citește JSON-ul și construiește un obiect **SignupRequest**.

SignupRequest: forma datelor de intrare

backend/src/main/java/com/netex/addressbook/auth/signup/SignupRequest.java · liniile 8-17

```
8 public record SignupRequest(  
9     @NotBlank @Email @Size(max = 254) String email,  
10    @NotBlank @Size(min = 8, max = 72) String password) {  
11  
12    public SignupRequest {  
13        if (email != null) {  
14            email = EmailAddress.normalize(email);  
15        }  
16    }  
17 }
```

- **record** este o formă compactă de clasă pentru date; aici are două componente: email și password. Nu reprezintă un rând din SQL.
- **@NotBlank** refuză lipsa sau un text format numai din spații. **@Email** verifică forma adresei. **@Size** limitează lungimea.
- Constructorul compact normalizează emailul înainte să ruleze validarea: elimină spațiile de la capete și transformă literele în lowercase.

EmailAddress: normalizarea comună

backend/src/main/java/com/netex/addressbook/auth/EmailAddress.java · liniile 5-13

```
5 public final class EmailAddress {  
6  
7     private EmailAddress() {  
8     }  
9  
10    public static String normalize(String email) {  
11        return email.trim().toLowerCase(Locale.ROOT);  
12    }  
13 }
```

Locale.ROOT face transformarea literelor independentă de limba computerului. Metoda este folosită și la login și la inițializarea contului admin; regula stă într-un singur loc.

Nu confunda normalizarea cu validarea

Normalizarea schimbă forma unei valori acceptabile: **'ANA@EXAMPLE.COM'** devine **'ana@example.com'**.

Validarea decide dacă valoarea este permisă: **'ana'** nu este un email valid.

4. Controllerul: ușa HTTP

backend/src/main/java/com/netex/addressbook/auth/signup/SignupController.java · liniile 12-27

```
12 @RestController
13 @RequestMapping("/api/auth/signup")
14 public class SignupController {
15
16     private final SignupService signupService;
17
18     public SignupController(SignupService signupService) {
19         this.signupService = signupService;
20     }
21
22     @PostMapping
23     public ResponseEntity<UserResponse> signup(@Valid @RequestBody SignupRequest request) {
24         return ResponseEntity.status(HttpStatus.CREATED)
25             .body(UserResponse.from(signupService.signup(request)));
26     }
27 }
```

Linie / concept	Ce înseamnă
@RestController	Spring folosește această clasă pentru răspunsuri HTTP; obiectul întors devine JSON.
@RequestMapping	Prefixul rutei este /api/auth/signup.
Constructor	Spring îi oferă un SignupService. Controllerul nu îl creează cu new.
@PostMapping	Metoda răspunde la POST pe ruta clasei.
@RequestBody	Spring convertește corpul JSON în SignupRequest.
@Valid	Spring aplică adnotările de validare de pe SignupRequest.
ResponseEntity	Stabilește explicit răspunsul: status 201 Created și corp UserResponse.

Dacă **@Valid** refuză emailul sau parola, metoda din controller nu ajunge să cheme Service-ul și clientul primește HTTP 400. Dacă datele sunt valide, controllerul delegă munca la **signupService.signup(request)**.

De ce există Service-ul separat?

Controllerul nu decide rolul, nu calculează hashul și nu scrie SQL. Astfel regulile pot fi citite și testate fără să fie amestecate cu detaliile HTTP.

5. Service-ul: regulile înregistrării

backend/src/main/java/com/netex/addressbook/auth/signup/SignupService.java · liniile 13-41

```
13  @Service
14  public class SignupService {
15
16      private final UserRepository users;
17      private final PasswordEncoder passwordEncoder;
18
19      public SignupService(UserRepository users, PasswordEncoder passwordEncoder) {
20          this.users = users;
21          this.passwordEncoder = passwordEncoder;
22      }
23
24      public AppUser signup(SignupRequest request) {
25          String email = EmailAddress.normalize(request.email());
26          if (email.isEmpty() || email.length() > 254 || !email.contains("@")) {
27              throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Invalid email");
28          }
29          if (request.password().getBytes(StandardCharsets.UTF_8).length > 72) {
30              throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Password is too long");
31          }
32
33          String hash = passwordEncoder.encode(request.password());
34          try {
35              long id = users.create(email, hash, "USER");
36              return new AppUser(id, email, hash, "USER");
37          } catch (DuplicateKeyException exception) {
38              throw new ResponseStatusException(HttpStatus.CONFLICT, "Email is already registered",
39                  exception);
40          }
41      }
42  }
```

Citește metoda **signup** de sus în jos. Primește un **SignupRequest**, obține emailul normalizat, face o verificare suplimentară și transformă parola în hash. Apoi cere Repository-ului să insereze contul cu rolul fix **USER**.

Observă că **SignupRequest** a normalizat deja emailul. Apelul repetat din Service nu schimbă rezultatul; este o mică redundanță din codul actual, nu un mecanism Spring separat.

Fragment	De ce există
<code>getBytes(UTF_8).length > 72</code>	BCrypt folosește cel mult 72 de octeți; evităm acceptarea unei parole a cărei parte finală ar fi ignorată.
<code>passwordEncoder.encode(...)</code>	Generează hashul. Parola în clar nu ajunge în INSERT.
<code>users.create(..., "USER")</code>	Rolul vine de la server. Clientul nu poate alege ADMIN la signup.
<code>DuplicateKeyException</code>	Indexul unic SQL a refuzat emailul deja existent; API-ul răspunde 409 Conflict.
<code>new AppUser(...)</code>	Construiește rezultatul intern cu ID-ul primit de la PostgreSQL.

Un lucru important despre hash

Hashul nu se decriptează. La login, Spring aplică verificarea parolei introduse față de hashul salvat. Două hashuri BCrypt pentru aceeași parolă pot arăta diferit, deoarece folosesc un salt.

6. Repository-ul: operația SQL

backend/src/main/java/com/netex/addressbook/auth/UserRepository.java · liniile 7-30

```

7  @Repository
8  public class UserRepository {
9
10     private final JdbcTemplate jdbc;
11
12     public UserRepository(JdbcTemplate jdbc) {
13         this.jdbc = jdbc;
14     }
15
16     public Optional<AppUser> findByEmail(String email) {
17         return jdbc.query("""
18             SELECT id, email, password_hash, role FROM users WHERE lower(email) = lower(?)
19             """, (rs, rowNum) -> new AppUser(
20                 rs.getLong("id"), rs.getString("email"),
21                 rs.getString("password_hash"), rs.getString("role")), email)
22             .stream().findFirst();
23     }
24
25     public long create(String email, String passwordHash, String role) {
26         return jdbc.queryForObject("""
27             INSERT INTO users (email, password_hash, role) VALUES (?, ?, ?) RETURNING id
28             """, Long.class, email, passwordHash, role);
29     }
30 }
```

- **@Repository** marchează componenta care accesează datele; Spring îi furnizează **JdbcTemplate** prin constructor.
- **create** trimite către PostgreSQL un **INSERT** cu email, hash și rol. **RETURNING id** cere bazei ID-ul nou generat.
- Cele trei caractere **?** sunt locuri pentru valori. **JdbcTemplate** le trimite ca parametri, astfel emailul nu este lipit în textul SQL.
- **findByEmail** citește un cont pentru loginul viitor; **Optional** exprimă faptul că emailul poate să nu existe.

Ce este AppUser și ce este UserResponse?

backend/src/main/java/com/netex/addressbook/auth/AppUser.java · liniile 1-4

```

1  package com.netex.addressbook.auth;
2
3  public record AppUser(long id, String email, String passwordHash, String role) {
4  }
```

backend/src/main/java/com/netex/addressbook/auth/UserResponse.java · liniile 1-12

```

1  package com.netex.addressbook.auth;
2
3  public record UserResponse(long id, String email, String role) {
4
5     public static UserResponse from(AppUser user) {
6         return new UserResponse(user.id(), user.email(), user.role());
7     }
8
9     public static UserResponse from(AccountPrincipal principal) {
10         return new UserResponse(principal.id(), principal.email(), principal.role());
11     }
12 }
```

AppUser este forma internă: include **passwordHash**, deoarece loginul va avea nevoie de el. **UserResponse** este forma publică: conține numai **id**, **email**, **role**. Metoda **from(AppUser)** face conversia. De aceea hashul nu apare în JSON-ul de răspuns.

7. Rezultate posibile și limitele acestui pas

Răspuns	Când apare în fluxul actual
201 Created	Contul a fost creat; JSON cu id, email și role USER.
400 Bad Request	Email/parolă invalide sau parola depășește 72 de octeți UTF-8.
403 Forbidden	Lipsește tokenul CSRF la POST, chiar dacă signup este public.
409 Conflict	Emailul există deja; indexul unic din PostgreSQL a refuzat INSERT-ul.

Public înseamnă că nu trebuie să fii logat pentru a crea un cont. Nu înseamnă că putem omite tokenul CSRF de la cererea POST. Pentru exercițiul practic îl obținem de la **GET /api/auth/csrf**; mecanismul îl vom studia în pasul 3.

Ce nu se întâmplă la signup

Înregistrarea **nu** autentifică automat utilizatorul. După răspunsul 201, **GET /api/auth/me** răspunde 401 până când utilizatorul face login. Evenimentul Kafka cerut de proiect este planificat pentru o etapă ulterioară; nu pretendem că este deja trimis.

Teste relevante din proiect

În **AuthApiTest**, testul **signupNeedsCsrfCreatesOnlyAUserAndNeverReturnsAHash** verifică 403 fără token, 201 cu token, rol USER chiar dacă cererea conține role ADMIN, absența hashului din JSON, existența hashului BCrypt în SQL și 409 la email duplicat. **invalidSignupsRejected** verifică 400. **DatabaseSchemaTest** verifică migrarea V2 și constrângerea rolului.

Testele folosesc un PostgreSQL temporar pornit de Testcontainers. Nu modifică baza locală **netex** pe care o vezi în DBeaver.

8. Exercițiu practic: creează un cont de test

Durată: aproximativ 15 minute. Ai nevoie de Docker Desktop, PostgreSQL pornit din Compose și backendul actual repornit pe portul 8080. Folosește o parolă de test, nu o parolă personală.

A. Pornește serviciile

În rădăcina repository-ului, rulează `docker compose up -d --wait postgres`. Apoi pornește backendul din IntelliJ cu profilul **local**, sau din directorul **backend** cu comanda din README. Dacă portul 8080 este ocupat de versiunea veche, oprește acea instanță și repornește backendul actual.

B. Trimite cererea din PowerShell

```
1 $session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
2 $csrf = Invoke-RestMethod http://localhost:8080/api/auth/csrf -WebSession $session
3 $headers = @{}
4 $headers[$csrf.headerName] = $csrf.token
5 $email = 'student-' + (Get-Random -Minimum 10000 -Maximum 99999) + '@example.com'
6 $body = @{ email = $email; password = 'ParolaDeTest123!' } | ConvertTo-Json
7 $result = Invoke-WebRequest http://localhost:8080/api/auth/signup -Method Post -WebSession
8           $session -Headers $headers -ContentType 'application/json' -Body $body
9 $result.StatusCode
10 $result.Content
```

Ar trebui să vezi **201** și un JSON cu ID, emailul generat și **role: USER**. Copiază emailul dacă vrei să folosești contul de test la pasul 2. Dacă primești 403, verifică faptul că ai folosit același **\$session** și headerul CSRF. Dacă primești 404 la `/api/auth/csrf`, backendul de pe 8080 este probabil versiunea veche.

C. Verifică vizual în DBeaver

- Dă Refresh la conexiunea bazei **netex**. Deschide **Schemas** → **public** → **Tables** → **users** → **View Data** → **All Rows**.
- Caută valoarea din **\$email**. Vezi **id**, emailul normalizat, **role = USER** și **created_at**.
- **password_hash** trebuie să fie diferit de parola de test și să înceapă, de regulă, cu **\$2**. Nu copia acest hash într-un mesaj public.
- În **flyway_schema_history**, verifică versiunile **1** și **2**, ambele reușite.

9. Consolidare: poți explica singur?

Închide PDF-ul pentru 5 minute. Deschide fișierele în IntelliJ și răspunde cu voce tare, în această ordine. Dacă nu știi un răspuns, revino doar la secțiunea respectivă.

1. Unde este definit tabelul users și de ce există o migrare V2?
2. Ce primește SignupController și ce îi transmite lui SignupService?
3. Cine alege rolul USER? Ce se întâmplă dacă browserul trimite ADMIN?
4. De ce UserRepository folosește ? în INSERT și RETURNING id?
5. Unde ajunge parola în clar? Unde ajunge hashul?
6. De ce JSON-ul de răspuns nu conține password_hash?
7. De ce poate apărea 409 chiar dacă Java ar verifica în prealabil existența emailului?

Răspunsul complet de interviu, în 30 de secunde

Model de explicație

La signup, Spring primește JSON-ul în SignupController și validează SignupRequest. SignupService normalizează emailul, verifică lungimea parolei în octeți, produce un hash BCrypt și impune rolul USER. UserRepository execută un INSERT parametrizat, iar PostgreSQL generează ID-ul și garantează emailul unic. Controllerul întoarce 201 cu UserResponse, care nu include hashul. Flyway ține structura users în V1 și rolul în V2.

Ce să cauți dacă vrei să aprofundezi

- Java: **record**, constructor, **Optional** și diferența dintre obiect și rând SQL.
- Spring: **@RestController**, **@RequestBody**, **@Valid**, injecție prin constructor, **JdbcTemplate**.
- Date și securitate: **Flyway migration**, **PostgreSQL unique index**, **BCrypt PasswordEncoder**.

Nu ai nevoie să studiezi acum implementarea internă a fiecărei adnotări. Obiectivul este să poți urmări traseul cererii și să explici de ce fiecare limită există.